Najm Eddine Zaga Follow
Feb 6 · 14 min read · Listen

Save

# 18 Best Practices for React

> *Code it better*

Open in app
Get unlimited access
Search Medium

React is one of the most popular techs when talking about JavaScript. Even for some developers, it's the best. React doesn't force you how to structure your project. It's completely up to you to choose how do it. As a result, that makes some developers

bring the artistic sense out of them. But, on the other hand… others can make a mess. When working as a team, it's better to make an understandable structure and code.

In this article you'll explore 18 tips about how to code better in React.

## 1 — Avoid local state as much as possible

In case you have some calculations, avoid putting the result in a local state and rendering that state later in the JSX. Instead, move your calculations in the JSX, or create a function that returns the result and put it in the JSX.

```
import React, { useEffect, useState } from 'react';

const App: React.FC = () => {
  // ❌ Avoid: Unnecessary state
  const [result, setResult] = useState();

  // Considering a and b are two values coming from some API.
  const { a, b } = apiCall();
  // ❌ Avoid: Unnecessary useEffect
  useEffect(() => {
    setResult(a + b);
  }, [a, b]);

  return (
    <div>
      {result}
    </div>
  );
}

export default App;
```

543 7

### Do this

```
import React, { useEffect, useState } from 'react';

const App: React.FC = () => {

  // Considering a and b are two values coming from some API.
```

```
  const { a, b } = apiCall();
  // ✅ Good: You can move it into the JSX
  return (
    <div>
      {a + b}
    </div>
  );
}

export default App;
```

## 2 — Use Functional components instead of Class components

To understand the use of Functional components, we should first get to know the difference between Functional and Class component.

Function component is just a plain JavaScript function which accepts props and returns a React element. On the other hand, a Class component is a class that extends from React.Component and creates a Render function that returns a React element. Obviously, this requires more code to write. With React Hooks, now you can write your components as functions instead of classes.

One of the major reasons to drop Class components, in Function components there is no *this* binding. Besides, they are easy to read, test and maintain.

### Class component

```
          this.setState({
            data: users,
          });
        });
    }

    render() {
      return (
        <ul>
          { this.state.usersData.map(user => <li key={user.id}>{user.name}</li>) }
        </ul>
      )
    }
  }
```

### The same thing with Functional component

```
import React, { useEffect, useState } from 'react';
// ✅ Good: Less code
export const LoadData: React.FC = () => {

  const [usersData, setUsersData] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((users) => {
        setUsersData(users);
      });
  }, []);

  return (
    <ul>
      {usersData.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  )
}
```

The difference is obvious.

## 3 — Use PropTypes

If you're not using TypeScript, consider using PropTypes. They're used to define the type of props in a component.

```
import PropTypes from 'prop-types';
import React from 'react';

export const SimpleComponent: React.FC = ({ name, age }) => {

  return (
    <div>My name is {name}, and I'm {age}</div>
  )
}
// ✅ Good: Each prop is well typed
SimpleComponent.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number.isRequired
}
```

Read more about Typechecking with PropTypes here.

## 4 — Composition

The React team suggests using a single responsibility principle. This means that one component should only be responsible for one functionality. If some of the components have more than one functionality, you should refactor and create a new component for every functionality.

Let's take this example:

```
    return (
      <div>
        <h2>{title}</h2>
        <h4>{author}</h4>
        <h4>Published the {date}</h4>
        <p>{body}</p>

        <div>Comments</div>
        <ul>
          {
            comments.map((comment) => <li key={comment.id}>{comment.content}</li>)
          }
        </ul>
        <div className='divider' />

        <div>Leave a comment</div>
        <form>
          <textarea name="" id="" cols="30" rows="10" onChange={handleTextAreaChange
          <button type="submit">Post comment</button>
        </form>
      </div>
    )
  }
```

This is a horrible way to use a component based library or framework. The cleanest way to do this is by dividing this component:

### ArticleContent component

```
import React from 'react';
// ✅ Good: Small, maintainable and easy to read
export const ArticleContent: React.FC = () => {

  // This is a stupid line to indicates that article is loaded by some API.
  const { title, author, body, date } = api.getArticle();

  return (
    <div>
      <h2>{title}</h2>
      <h4>{author}</h4>
      <h4>Published the {date}</h4>
      <p>{body}</p>
    </div>
```

```
    )
  }
```

### CommentSection component

```
import React from 'react';
// ✅ Good: Small, maintainable and easy to read
export const CommentSection: React.FC = () => {

  // This is a stupid line to indicates that article is loaded by some API.
  const { comments } = api.getArticle();

  return (
    <div>
      <div>Comments</div>
      <ul>
        { comments.map((comment) => <li key={comment.id}>{comment.content}</li>) }
      </ul>
    </div>
  )
}
```

### CommentForm component

```
    )
  }
```

**Article component**

```
import React from 'react';
import ArticleContent from './components/ArticleContent';
import CommentSection from './components/CommentSection';
import CommentForm from './components/CommentForm';
// ✅ Good: Small, maintainable and easy to read
export const Aricle: React.FC = () => {

  return (
    <div>
      <ArticleContent />
      <CommentSection />
      <div className='divider' />
      <CommentForm />
    </div>
  )
}
```

## 5 — Keep components small

Small components are easy to read, test, maintain and re-use. This tip is should be used with previous one '**Composition**'.

## 6 — Consider using TypeScript

TypeScript will guarantee a better static typing. As we know, Javascript is a dynamic-typing language. That means you can define a variable and assign a boolean value, then override it later with a string value. This will crush your app at some point. TypeScript will ensure that your variables will only accept the types you defined.

```
  id: string;
  content: string;
  date: Date;
}

interface IArticle {
  title: string;
  author: string;
  date: Date;
  body: string;
  views: number;
  comments: IComment[];
}

export const Aricle: React.FC<IArticle> = ({ title, author, date, body, views, co

  const [variable, setVariable] = useState<IArticle | null>(null)

  return (
    <div>
      <h2>{title}</h2>
      <h4>{author}</h4>
      <h4>Published the {date}</h4>
      <p>{body}</p>
    </div>
  )
}
```

## 7 — Avoid using index as key prop

React uses keys specifically to identify items in Array. Using the key prop, React can track which item has changed, item has been added/removed to/from the Array.

Okay, so why we should avoid indexes ?

Indexes are re-generated on every render… If any item has been changed, React will never identify which element has been changed. Why ? Because:

```
// Array:   [10, 20, 30, 40, 50]
// Indexes:  0    1   2   3   4
```

Lets consider that we removed the first element which is "**10**", in this case indexes will be like:

```
// Array:   [20, 30, 40, 50]
// Indexes:  0    1   2   3
```

For React, all elements keys has changed, because their not in the same position they were.

As a result, consider the following code is a very bad practice:

```
// ❌ Avoid
<ul>
  { comments.map((comment, index) => <li key={index}>{comment.content}</li>) }
</ul>
```

Instead, use a unique value like "*id*" to make sure the key will never change.

```
// ✅ Good
<ul>
  { comments.map((comment, index) => <li key={comment.id}>{comment.content}</li>)
</ul>
```

## 8 — Consider using React Fragments

In React, a component must return a single element. That element can be a wrapper for some children elements and so on.

But, what if I don't need to render the parent element, and only children are needed to be present in **DOM** tree!

The first reaction will be… Okay, wrap them with a ** ** with no **className**… Okay this will do the job. But, it will increase the size of DOM. When working with a massive project, more DOM nodes means more memory needed.

## 9 — Prefixing variables and methods

When coding a huge project, or even a smaller one. Consider using some prefixes to make track easier.

- **'is'** and **'has'** prefixes can be used with boolean typed variables, this should tell **'this variable is boolean typed'**. Even a method can be prefixed with **'is'** or **'has'** to indicate that it returns a boolean value.

```
import { Modal } from 'antd';
import React, { useState } from 'react';
import SomeComponent from 'somewhere';

export const Aricle: React.FC = () => {

  const [isOpen, setOpen] = useState(false);
  const [hasParent, setParent] = useState(false);

  return (
    <div>
      <Modal open={isOpen} />
      <SomeComponent parent={hasParent} />
    </div>
  )
}
```

- **handle** and **on** prefixes should be used with methods only to make easier recognizing it's a method, and what kind of method.

### — handle

Should tell that a method will be passed to an event listener, and will be invoked once the event triggered:

```
import { Modal } from 'antd';
import React, { useState } from 'react';

export const Aricle: React.FC = () => {

  const [isOpen, setOpen] = useState(false);

  // Indicates that a method is used as 'event callback'
  const handleToggleModal = () => setOpen((prev) => !prev);

  return (
    <div>
      <Modal open={isOpen} />
      <button onClick={handleToggleModal}>Toggle show</button>
    </div>
  )
}
```

**— on**

Usually, ‘**on**’ should be a prefix to a prop name when passing a method as a callback to another component. Received props can be of different types. The ‘**on**’ prefix tells this is a callback just by reading the prop name.

```
import React from 'react';
import Form from './components/Form';

export const CreateUser: React.FC = () => {

  const handleFormSubmit = () => {
    // Send data
  };

  return (
    <div>
      <Form onFormSubmit={handleFormSubmit} />
    </div>
  )
}
```

```
import React from 'react';

export const Form: React.FC = ({ onFormSubmit }) => {

  return (
    <form onSubmit={onFormSubmit}>
      {/* Some inputs */}
      <button type="submit">Create User</button>
    </form>
  )
}
```

## 10 — Keep your code DRY

Don't Repeat Yourself, keep in mind whenever you have to duplicate the code, just create a method you can call instead of duplicating the code. That's what we call a modular code.

```
// ❌ Avoid: Redundant code
const cyberSecTotalStudents = 80;
const myCyberSecClassStudents = 24;

const AITotalStudents = 150;
const myAIClassStudents = 24;

const cyberSecPercentage = (myCyberSecClassStudents / cyberSecTotalStudents) * 10
const AIPercentage = (myAIClassStudents / AITotalStudents) * 100;
```

```
const cyberSecPercentage = calculatePercentage(myCyberSecClassStudents, cyberSecT
const AIPercentage = calculatePercentage(myAIClassStudents, AITotalStudents);
```

## 11 — Use ES6 destructuring for your props

Avoid using objects as props inside your code. When using objects inside your code will refer to the props object whenever that object is used.

```
import React from 'react';
// ❌ Avoid: Refer to props object
export const Form: React.FC = (props) => {

  return (
    <div>
      <h2>Profile</h2>
      {props.name}
    </div>
  )
}
```

Instead, you can use ES6 destructuring to avoid that.

```
import React from 'react';
// ✅ Good: Refer to text
export const Form: React.FC = ({ name }) => {

  return (
    <div>
      <h2>Profile</h2>
      {name}
    </div>
  )
}
```

## 12 — Avoid anonymous functions in your HTML

A JavaScript function is a block of code designed to perform a specific task. When defining a function, a memory space is used to store it.

### Anonymous function

```
import React from 'react';

export const Form: React.FC = () => {

  return (
    <form onSubmit={
      // ❌ Avoid: re-created on every render
      () => {
        // Send data
      }}
    >
      {/* Some inputs */}
      <button type="submit">Send</button>
    </form>
  )
}
```

### Named function

```
import React from 'react';

export const Form: React.FC = () => {
  // ✅ Good: Loaded in memory
  const handleFormSubmit = () => {
    // Send data
  };

  return (
    <form onSubmit={handleFormSubmit}>
      {/* Some inputs */}
      <button type="submit">Send</button>
    </form>
  )
}
```

Both functions do the same job. But… as it's located in a memory space, when called, a named function is called by its reference in memory. On the other hand, anonymous functions are not memorized. They get re-created on every render. This means, a different function is used for each render.

### 13 — Consider passing a callback argument to useState setter method

When using a state setter method to update the state value based on the previous value, you'll need to get the previous value by calling the state variable like:

```
import React, { useState } from 'react';

export const MakeSum: React.FC = () => {

  const [sum, setSum] = useState(0);
  // ❌ Avoid
  const handleIncreaseValue = () => setSum(sum + 1);
  const handleDecreaseValue = () => setSum(sum - 1);

  return (
    <>
      <button onClick={handleIncreaseValue}> +1</button>
      <button onClick={handleDecreaseValue}> -1</button>
    </>
  )
}
```

This is a bad reaction… If you're updating the previous state value, you should pass a function as an argument to the state setter instead:

```
import React, { useState } from 'react';

export const MakeSum: React.FC = () => {

  const [sum, setSum] = useState(0);
  // ✅ Good
  const handleIncreaseValue = () => setSum((previousState) => {
    return previousState + 1;
  });
  const handleDecreaseValue = () => setSum((previousState) => {
```

```
      return previousState - 1;
    });

    return (
      <>
        <button onClick={handleIncreaseValue}> +1</button>
        <button onClick={handleDecreaseValue}> -1</button>
      </>
    )
  }
```

Both ways work just the same, but… When you have multiple state setters in a row, React will batch them together and run them all at the same time, and the state value will never change the way you expect:

```
import React, { useState, useEffect } from 'react';

export const MakeSum: React.FC = () => {

  const [sum, setSum] = useState(0);

  const handleIncreaseValue = () => {
    // ❌ Avoid: Will be batched and causes issues
    setSum(sum + 1);
    setSum(sum + 1);
    setSum(sum + 1);
  }

  useEffect(() => {
    console.log(sum);
    // Expected output: 3
    // Result outpul: 1
  }, [sum]);

  return (
    <>
      <button onClick={handleIncreaseValue}> +1</button>
    </>
  )
}
```

```
package.json ×   index.html ×   style.css ×   App.jsx ×   index.jsx ×
1  import React, {useState, useEffect} from 'react';
2
3  export function App(props) {
4    const [sum, setSum] = useState(0);
5    const handleIncreaseValue = () => {
6      setSum(sum + 1);
7      setSum(sum + 1);
8      setSum(sum + 1);
9    }
10   useEffect(() => {
11     console.log(sum);
12   }, [sum]);
13   return (
14     <div className='App'>
15       <h1>Sum is: {sum}</h1>
CONSOLE ×                                   WEBSITE VIEW ×
0
                                            Sum is: 0
                                            +1
```

This is the initial output before increasing the sum

```
package.json ×   index.html ×   style.css ×   App.jsx ×   index.jsx ×
1  import React, {useState, useEffect} from 'react';
2
3  export function App(props) {
4    const [sum, setSum] = useState(0);
5    const handleIncreaseValue = () => {
6      setSum(sum + 1);
7      setSum(sum + 1);
8      setSum(sum + 1);
9    }
10   useEffect(() => {
11     console.log(sum);
12   }, [sum]);
13   return (
14     <div className='App'>
15       <h1>Sum is: {sum}</h1>
CONSOLE ×                                   WEBSITE VIEW ×
0
1
                                            Sum is: 1
                                            +1
```

Output after one click on the +1 button

Let’s try the same code, but passing a method instead:

```
package.json ×  index.html ×  style.css ×  App.jsx ×  index.jsx ×
1  import React, {useState, useEffect} from 'react';
2
3  export function App(props) {
4    const [sum, setSum] = useState(0);
5    const handleIncreaseValue = () => {
6      setSum((prevValue) => prevValue + 1);
7      setSum((prevValue) => prevValue + 1);
8      setSum((prevValue) => prevValue + 1);
9    }
10   useEffect(() => {
11     console.log(sum);
12   }, [sum]);
13   return (
14     <div className='App'>
15       <h1>Sum is: {sum}</h1>
CONSOLE ×                                  WEBSITE VIEW ×
0
                                           Sum is: 0
                                           +1
```

This is the initial output before increasing the sum

```
package.json ×   index.html ×   style.css ×   App.jsx ×   index.jsx ×
1  import React, {useState, useEffect} from 'react';
2
3  export function App(props) {
4    const [sum, setSum] = useState(0);
5    const handleIncreaseValue = () => {
6      setSum((prevValue) => prevValue + 1);
7      setSum((prevValue) => prevValue + 1);
8      setSum((prevValue) => prevValue + 1);
9    }
10   useEffect(() => {
11     console.log(sum);
12   }, [sum]);
13   return (
14     <div className='App'>
15       <h1>Sum is: {sum}</h1>
```

```
CONSOLE ×
0
3
```

WEBSITE VIEW ×

Sum is: 3

+1

Output after one click on the +1 button

When using a callback, the setter function will pass the actual value of the state as an argument to the callback method, for each setter call we're receiving the new value of the state.

## 14 — Use JSX / TSX extension for your files

If you would like to use **Emmet** in React, consider using the **.jsx** / **.tsx** extension when creating components.

## 15 — Don't create unnecessary Effects

A **useEffect** hook is needed to synchronize with external systems. Like fetching/sending data from/to external source.

**UseEffect** hook has 3 effects:

- **ComponentDidMount** — When the component is mounted and being rendered in the browser's DOM tree.

- **ComponentDidUpdate** — When the component re-render due to a state/props update.
- **ComponentWillUnmout** — When the component is about to unmout (before being removed from browser's DOM tree).

According to the docs in *beta.reactjs.org*, you might not need an Effect. Usually, we need the **useEffect** hook to do something when a specific or a group of states or props are updated.

```
function Form() {
  const [firstName, setFirstName] = useState('Taylor');
  const [lastName, setLastName] = useState('Swift');

  // ❌ Avoid: redundant state and unnecessary Effect
  const [fullName, setFullName] = useState('');

  useEffect(() => {
    setFullName(firstName + ' ' + lastName);
  }, [firstName, lastName]);
  // ...
}
```

```
function Form() {
  const [firstName, setFirstName] = useState('Taylor');
  const [lastName, setLastName] = useState('Swift');
  // ✅ Good: calculated during rendering
  const fullName = firstName + ' ' + lastName;
  // ...
}
```

When having expensive calculations, don't feel necessary to store results in a state:

```
function TodoList({ todos, filter }) {
  const [newTodo, setNewTodo] = useState('');

  // ❌ Avoid: redundant state and unnecessary Effect
```

```
  const [visibleTodos, setVisibleTodos] = useState([]);
  useEffect(() => {
    setVisibleTodos(getFilteredTodos(todos, filter));
  }, [todos, filter]);

  // ...
}
```

Keep it simple:

```
function TodoList({ todos, filter }) {
  const [newTodo, setNewTodo] = useState('');
  // ✅ This is fine if getFilteredTodos() is not slow.
  const visibleTodos = getFilteredTodos(todos, filter);
  // ...
}
```

You can use memoization in some cases with **useMemo** hook:

```
import { useMemo, useState } from 'react';

function TodoList({ todos, filter }) {
  const [newTodo, setNewTodo] = useState('');
  // ✅ Does not re-run getFilteredTodos() unless todos or filter change
  const visibleTodos = useMemo(() => getFilteredTodos(todos, filter), [todos, fil
  // ...
}
```

> *This tells React that you don't want the inner function to re-run unless either* `todos` *or* `filter` *have changed — reactjs docs.*

## 16 — Avoid unnecessary comments

When coding, lots of developers write lots of unnecessary comments:

```
// ❌ Avoid: Unnecessary comment
// This function filters fruits and return apples only
const handleDecreaseValue = (fruits) => {
  return fruits.filter(({ fruit }) => fruit === 'apple');
};
```

When reading the code, you should understand what it does without comments. If it's not clear enough… you should train yourself how to write readable code.

You can use comments only when documenting a reusable function:

```
/**
 *
 * @param {FileData} info: Uploaded files informations.
 * @param {function} setLoadingUpload: Loading state dispatcher.
 * @param {function} setImageUrl: Dispatch image url state.
 * @param {function} setFieldsValue: Dispatch image field value.
 * @param {function} fieldName: Field name in which will be dispatched.
 * @returns void
 */
export const handleFileChange = (info, setLoadingUpload, setImageUrl, setFieldValu
  if (info.file.status === 'uploading') {
    setLoadingUpload(true);
    return;
  }
  if (info.file.status === 'done') {
    // Get this url from response in real world.
    getBase64(info.file.originFileObj, (imageUrl) => {
      setLoadingUpload(false);
      setImageUrl(info.file.response.contentUrl);
      setFieldValue({
        [fieldName]: info.file.response["@id"],
        [showFileName]: info.file.name
      })
    });
  }
};
```

When invoking the function or hovering, you'll see this:

Functions documentation using better comment

## 17 — Don't mix between specific and shared components

Shared components are made to be re-usable across the application. Their main role is to fit wherever you want, based on props options:

```
import React from 'react';

export const CustomButton: React.FC = ({ text, bgColor, icon, onButtonClick }) =>
  const styles = {
    backgroundColor: bgColor,
  }
  return (
    <button style={styles} onClick={onButtonClick}>
      {icon}
      {text}
    </button>
  )
}
```

A specific component, is not a customizable component, it's created to be used in one single place in the app.

In this case, you should create a separate `/components` folder in the `/src` that contains re-usable components.

## 18 — Avoid treating pages as components

Many developers put pages components with the re-usable components in the same folder. This will not affect the application in term of functioning. But it's a bad practice in term of structuring and organizing files.

Pages are just create as components, but they should be in a separate folder that tells "**These components are considered as pages**".

## Conclusion

There are more best practices you can explore by yourself, and based on how you want to code, structure and organize a project. And there are some standards of how to do things better.

I hope this post was helpful. I tried to put the most important best practices I've learned over my years of experience with React. I'll have time to answer any question you put in the comment section below.

Happy coding!

Reference: https://beta.reactjs.org/

React JavaScript Best Practices React Hook